

**INSTITUTO POLITÉCNICO NACIONAL**

**Escuela Superior de Cómputo**

**ESCOM**




Materia: Bases de Datos

Profesor: Gabriel Hurtado Aviles

Grupo: 3CV4

**Título:**

# Práctica 1: "Modelo Entidad Relación"

**Miembros de Equipo:**

Jiménez Sánchez Taylin

## Introducción

A lo largo de esta práctica se resolverán algunos ejercicios para poner en marcha lo visto y aprendido sobre el sistema de git y github aplicándolo en varias áreas de interés propio y ejecutando de la misma forma que lo enseñado por el profesor.

# Práctica 1: "Modelo Entidad Relación"

Objetivo: Que usted describa los sistemas de base de datos a partir de sus aspectos básicos, construya el entorno de trabajo que empleará durante todo el semestre —control de versiones y un sistema gestor de bases de datos ejecutado en un contenedor—, sustente su investigación con literatura científica arbitrada y aplique lo anterior al modelado conceptual de un problema real. Toda la evidencia se entrega en un repositorio de Git.

Esta práctica cubre la Unidad Temática I completa y, al mismo tiempo, establece la forma de trabajo del resto del curso. Se divide en cinco ejercicios: los dos primeros construyen el entorno técnico, el tercero y el cuarto sustentan la parte conceptual, y el quinto la aplica a un caso concreto. La entrega no es un archivo suelto: es un repositorio de Git donde quedará registrado cómo trabajó usted, no solo el resultado final.

## Ejercicio 1. Control de versiones con Git y GitHub

Parte A. Investigación. Redacte al menos una cuartilla que responda lo siguiente:

1. Qué es un sistema de control de versiones y qué problema concreto resuelve en un trabajo en equipo.

Un sistema de control de versiones (VCS) es una herramienta informática que permite registrar y organizar los cambios que se realizan en los archivos de un proyecto a lo largo del tiempo. Su función principal es conservar un historial de las modificaciones para poder consultar o recuperar versiones anteriores cuando sea necesario.

2. Cuál es la diferencia entre Git y GitHub. No son sinónimos.

Git es un sistema de control de versiones distribuido que permite registrar, administrar y consultar los cambios realizados en el código de un proyecto. También facilita que varios programadores trabajen de manera independiente y posteriormente integren sus modificaciones.

Por otro lado, GitHub es una plataforma en línea que permite almacenar repositorios de Git y compartirlos con otros usuarios. Asimismo, ofrece herramientas para organizar proyectos, revisar código, proponer cambios y colaborar con otros desarrolladores. En resumen, Git se utiliza para controlar las versiones de los archivos, mientras que GitHub facilita su almacenamiento y la colaboración entre los integrantes de un equipo.

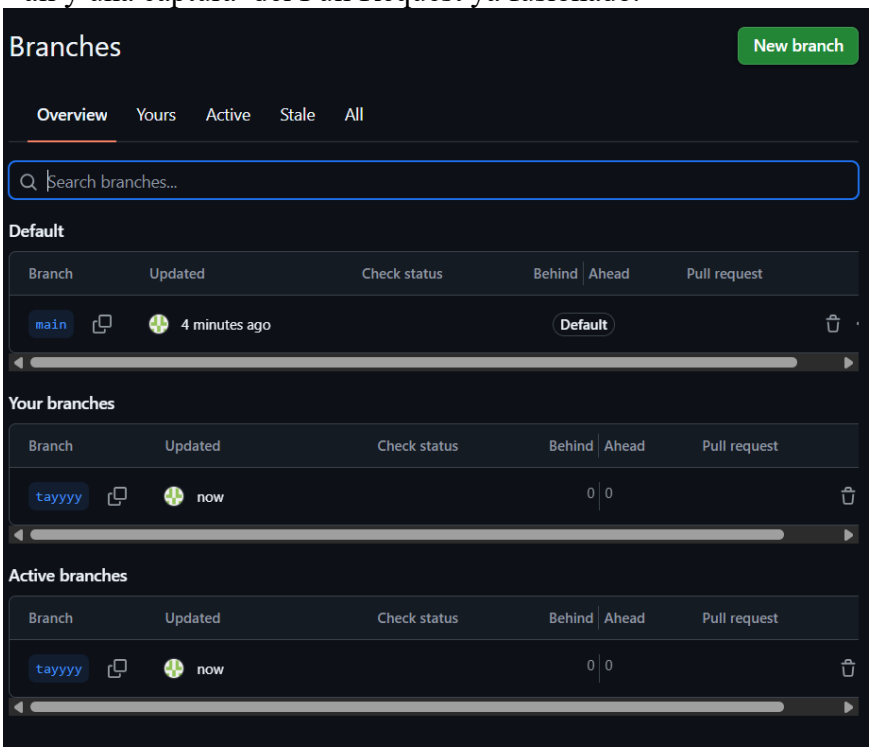
3. Defina, con sus palabras y con un ejemplo: repositorio, confirmación (commit), rama (branch), fusión (merge), conflicto de fusión, Pull Request, archivo .gitignore y archivo README.

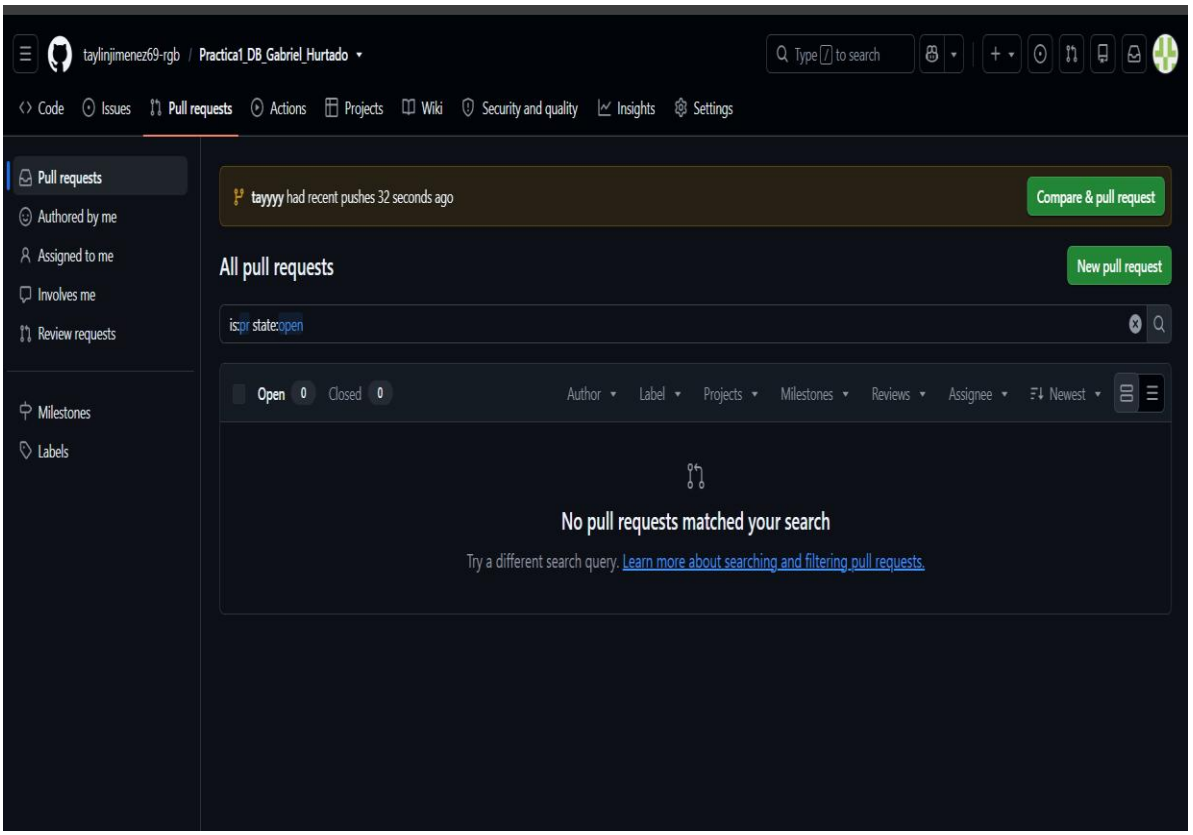
- **Repositorio:** Es el espacio donde se guardan los archivos de un proyecto junto con el historial de los cambios que se han realizado. Permite llevar un registro de las modificaciones hechas por los integrantes del equipo. Ejemplo: La carpeta sistema-escolar/, que contiene el código fuente y los archivos necesarios para administrar las calificaciones de los estudiantes.
- Confirmación (commit): Es un registro de los cambios realizados en un momento determinado. Cada confirmación cuenta con un identificador que permite reconocerla y consultar las modificaciones que contiene. Ejemplo: Ejecutar git commit -m "Agregar formulario de registro de alumnos" para guardar los cambios realizados.
- Rama (branch): Es una línea de trabajo independiente que permite desarrollar nuevas funciones o modificar partes de un proyecto sin afectar directamente la rama principal, llamada normalmente main o master. Ejemplo: Crear la rama login-usuarios para desarrollar la pantalla de inicio de sesión del sistema.
- Fusión (merge): Es el proceso mediante el cual se integran los cambios realizados en una rama con otra. Se utiliza para incorporar el trabajo terminado al código principal del proyecto. Ejemplo: Integrar los cambios de la rama login-usuarios en la rama main después de terminar y revisar el inicio de sesión.
- Conflicto de fusión: Ocurre cuando Git no puede combinar automáticamente determinados cambios, generalmente porque dos integrantes modificaron la misma parte de un archivo. En ese caso, es necesario revisar las diferencias y decidir qué modificaciones deben conservarse. Ejemplo: Dos compañeros cambian el mismo mensaje de error en un archivo y Git solicita resolver las diferencias antes de completar la fusión.
- Pull Request: Es una solicitud que se realiza para proponer la integración de los cambios de una rama en otra. Permite que los demás integrantes revisen las modificaciones, hagan observaciones y aprueben los cambios antes de incorporarlos. Ejemplo: Crear un Pull Request para solicitar que se revise y apruebe la nueva función de búsqueda de estudiantes antes de integrarla en main.
- Archivo .gitignore: Es un archivo de texto que contiene los nombres o patrones de los archivos y directorios que Git debe ignorar para no incluirlos en el seguimiento del proyecto. Esto ayuda a evitar que se agreguen archivos innecesarios. Ejemplo: Escribir node_modules/ en el archivo .gitignore para evitar que Git rastree esa carpeta de dependencias de un proyecto de Node.js.
- Archivo README: Es un documento que proporciona información importante sobre un proyecto, como su propósito, los requisitos necesarios, los pasos de instalación y las instrucciones para utilizarlo. Sirve como una guía para que otras personas comprendan cómo funciona el proyecto. Ejemplo: Crear un archivo README.md que explique cómo instalar y ejecutar una aplicación de inventario desarrollada en Python.

4. En qué consiste un flujo de trabajo basado en ramas y por qué el código se revisa entre pares antes de fusionarse.

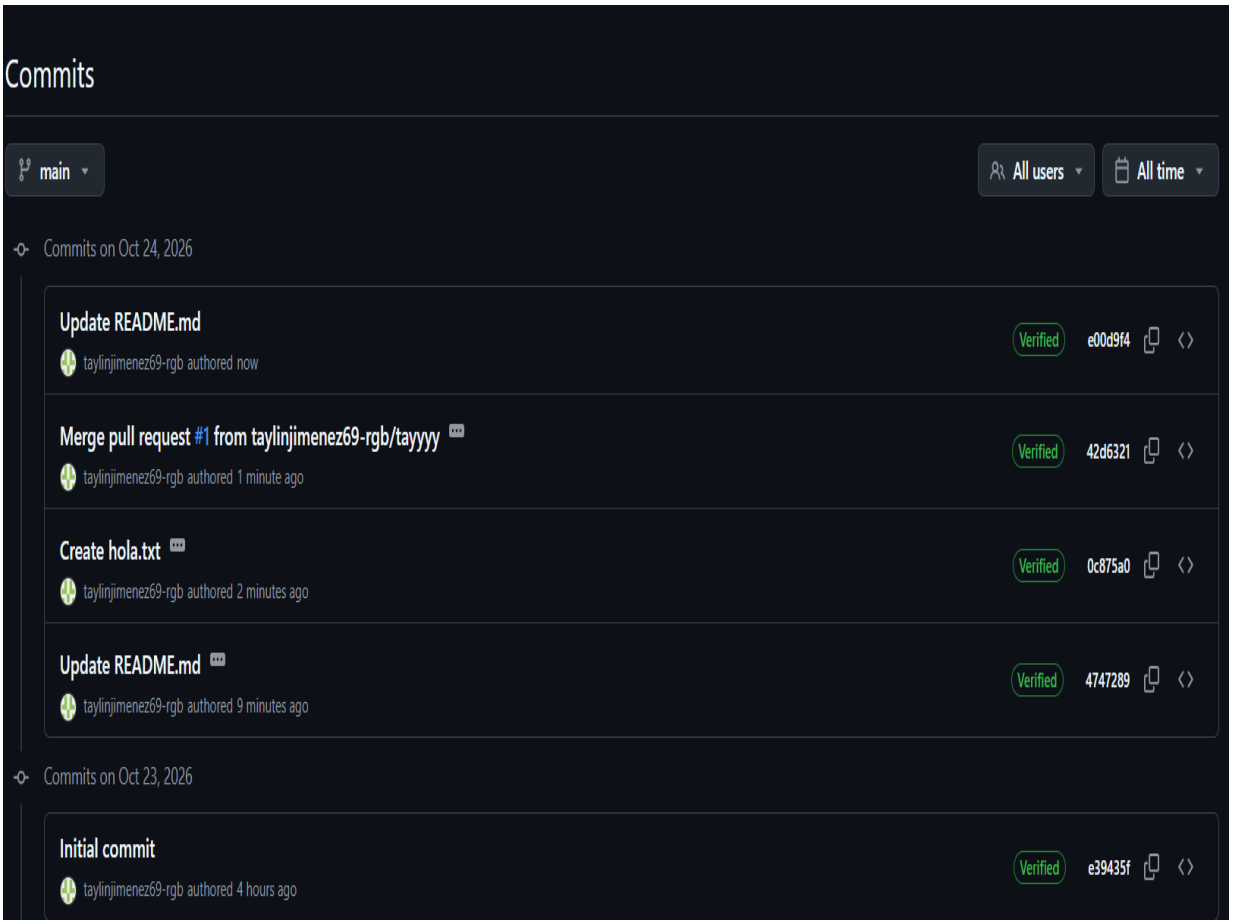
Un flujo de trabajo basado en ramas consiste en organizar el desarrollo de un proyecto mediante diferentes líneas de trabajo. Generalmente, la rama principal, conocida como main o master, se mantiene con una versión estable del código. Mientras tanto, las nuevas funciones, correcciones y pruebas se realizan en ramas secundarias, evitando modificar directamente la versión principal. Cuando los cambios están terminados y han sido revisados, pueden integrarse en la rama principal mediante una fusión.
La revisión entre pares consiste en que otros integrantes del equipo examinen el código propuesto antes de incorporarlo al proyecto. Este procedimiento normalmente se realiza mediante un Pull Request y permite detectar errores, mejorar la calidad del código, comprobar que los cambios funcionen correctamente y reducir posibles problemas de seguridad. Además, favorece el intercambio de conocimientos entre los miembros del equipo y ayuda a mantener un proyecto organizado, confiable y fácil de mantener.

Parte B. Práctica. Realice lo siguiente:
1. Cree una cuenta en GitHub, si aún no la tiene, y un repositorio para esta práctica.
2. Agregue un archivo README.md con su nombre completo, grupo, carrera y un índice de la práctica.
3. Agregue un archivo .gitignore adecuado al contenido del repositorio.
4. Realice al menos cinco confirmaciones con mensajes descriptivos. Un mensaje como «cambios» no cuenta.
5. Cree una rama, haga en ella un cambio, abra un Pull Request, descríbalo y fusiónelo.
Evidencia: la dirección del repositorio, una captura de la salida de git log --oneline --graph --all y una captura del Pull Request ya fusionado.




taylinjimenez69-rgb / Practica1_DB_Gabriel_Hurtado
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
Pull requests
Authored by me
Assigned to me
Involves me
Review requests
Milestones
Labels
tayyyy had recent pushes 32 seconds ago
Compare & pull request
All pull requests
New pull request
is:pr state:open
Open 0
Closed 0
Author
Label
Projects
Milestones
Reviews
Assignee
Newest
No pull requests matched your search
Try a different search query. Learn more about searching and filtering pull requests.


Practica1_DB_Gabriel_Hurtado
Public
Pin
main
2 Branches
0 Tags
Go to file
T
Add file
Code
taylinjimenez69-rgb Update README.md
e00d9f4 · now
5 Commits
README.md
Update README.md
now
hola.txt
Create hola.txt
1 minute ago
README


Commits
main
All users
All time
Commits on Oct 24, 2026
Update README.md
taylinjimenez69-rgb authored now
Verified
e00d9f4
Merge pull request #1 from taylinjimenez69-rgb/tayyyy
taylinjimenez69-rgb authored 1 minute ago
Verified
42d6321
Create hola.txt
taylinjimenez69-rgb authored 2 minutes ago
Verified
0c875a0
Update README.md
taylinjimenez69-rgb authored 9 minutes ago
Verified
4747289
Commits on Oct 23, 2026
Initial commit
taylinjimenez69-rgb authored 4 hours ago
Verified
e39435f

## Conclusión

Jimenez Sanchez Taylin:

En el desarrollo de esta primera práctica se logró construir y consolidar el entorno técnico de trabajo indispensable para la materia de Bases de Datos, a través del dominio de herramientas de control de versiones y metodologías de trabajo colaborativo, se logró estructurar un entorno de desarrollo ordenado, seguro y eficiente. La aplicación estructurada de ramas y revisiones de código no solo mitiga los riesgos de pérdida de información o conflictos en el repositorio, sino que inculca los estándares de calidad y rigor profesional exigidos en la industria tecnológica actual para la gestión de proyectos basados en datos.

## Bibliografía


Ambler, S. W., & Sadalage, P. J. (2006). Refactoring Databases: Evolutionary Database Design. Addison-Wesley Professional.

Coronel, C., & Morris, S. (2018). Database Systems: Design, Implementation, & Management (13.ª ed.). Cengage Learning.

Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9.ª ed.). McGraw-Hill Education.

Sadalage, P. J., & Fowler, M. (2012). NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley Professional.